Open Source Audit- Python

Name- Aryan Mauryakant

Reg. No.24BAI10142

Date- 30 March 2026

1. Introduction

Software is at the core of almost every system we interact with today, whether it is a mobile application, a website, or a large-scale enterprise platform. While many people are familiar with commercial software, a large portion of the digital world actually depends on open-source software. Open-source software is built collaboratively and is made available to everyone, allowing users not only to use it but also to study, modify, and improve it.

This project focuses on understanding open-source software in a deeper way. Instead of just looking at features, it examines the purpose behind the software, the problems it was designed to solve, the kind of freedom it offers, and the role it plays in the larger technological ecosystem.

For this audit, I have chosen Python as the software to study. Python is one of the most widely used programming languages today. It is known for its simplicity and readability, which makes it suitable for beginners as well as professionals. It is used in many fields such as web development, data science, artificial intelligence, automation, and cybersecurity.

The report is divided into several parts. The first part discusses the origin and philosophy behind Python, including its licensing and ethical aspects. The second part focuses on how Python exists and functions within a Linux system. The third part explores the ecosystem surrounding Python and how it connects with other open-source tools. The fourth part compares Python with proprietary alternatives. Finally, the project includes shell scripts that demonstrate practical knowledge of Linux and open-source tools.

Part A: Origin and Philosophy

A1. **The Problem Python Was Created to Solve**

Python was created by Guido van Rossum in the late 1980s and released in 1991. At that time, programming languages such as C and C++ were dominant. While these languages were powerful, they were also complex and required a deep understanding of low-level programming concepts. Writing even simple programs often required a lot of code, and debugging could be time-consuming.

One of the biggest issues developers faced was readability. Code written in these languages could be difficult to understand, especially for beginners. This made collaboration harder and increased the effort required to maintain software over time. Guido van Rossum wanted to create a language that would make programming more accessible and less complicated.

Python was designed with simplicity in mind. Its syntax is clean and easy to read, which reduces the effort required to understand code. This makes it easier for developers to work

together and maintain large projects. The language emphasizes readability, which is one of its most important features.

Another problem Python addressed was development speed. In many traditional languages, developers had to write a large amount of code to achieve simple tasks. Python allows developers to write fewer lines of code while still achieving the same results. This increases productivity and reduces development time.

Python was also designed to be flexible. It can be used for a wide range of applications, from simple scripts to complex systems. This versatility has contributed to its widespread adoption.

An important decision in Python's development was to make it open source. By doing so, it allowed developers around the world to contribute to its improvement. Over time, this collaborative approach helped Python evolve into one of the most widely used programming languages in the world.

From a personal perspective, Python stands out because of how easy it is to start using it. Even someone with basic programming knowledge can quickly write working code. This makes it a good choice for learning as well as for professional use. I personally found Python easier to work with compared to other languages I have used, especially because of its readability.

A2. **License Analysis**

Python is distributed under the Python Software Foundation License, which is a permissive open-source license. This type of license allows users to use, modify, and distribute the software with very few restrictions.

The idea of free software is based on four fundamental freedoms. These include the freedom to run the program for any purpose, the freedom to study how the program works, the freedom to modify it, and the freedom to distribute copies. Python satisfies all these freedoms, which makes it a true example of open-source software.

One important aspect of Python's license is that it allows commercial use. Companies can use Python in their products without being required to release their own source code. This is different from more restrictive licenses such as the GPL, which require derivative works to also be open source.

When comparing different licenses, the GPL focuses on ensuring that software remains open, while licenses like MIT and PSF focus on flexibility. Python's license is closer to the MIT license, which allows developers more freedom in how they use the software.

Another important concept is the difference between free software in terms of cost and freedom. Free software does not only mean that it is available at no cost. It also means that

users have control over how they use and modify the software. Python represents this idea of freedom because it allows users to fully interact with the software without restrictions.

If I were to choose a license for my own project, I would prefer a permissive license like MIT or PSF. This is because it allows more people to use the software freely, including companies, which can help increase adoption.

A3. **Ethics of Open Source**

Open-source software is not just about code; it also involves ethical considerations. One of the main benefits of open source is transparency. Since the source code is available, users can verify how the software works and ensure that it does not contain hidden functionality.

Open source also promotes collaboration. Developers from different backgrounds can contribute to the same project, which leads to better ideas and faster development. Python is a good example of this, as it has grown through contributions from a global community.

However, not all software can or should be open source. Some systems require strict control for security or privacy reasons. In such cases, keeping the source code private may be necessary.

Another important ethical question is whether companies should profit from open-source software. Many companies use Python to build products and generate revenue. While this may seem unfair at first, these companies often contribute back to the community in different ways, such as funding development or releasing their own tools.

The concept of standing on the shoulders of giants is very relevant in this context. Python itself was built on ideas from earlier programming languages, and now it serves as a foundation for new technologies. Open source allows this cycle of improvement to continue.

In my opinion, open source plays an important role in making technology accessible. It allows people from different backgrounds to learn and contribute, which leads to more innovation.

Part B: Linux Footprint

Python is an important part of most Linux systems and is often pre-installed.

Installation

Python can be installed using the package manager. The most common command is:

sudo apt install python3

```
C:\Users\Lenovo>python --version
Python 3.13.2
```

Key Directories

Python files are stored in different directories within the Linux system. The executable is usually located in /usr/bin, while libraries are stored in /usr/lib. Configuration files may be found in /etc.

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ which python3
/usr/bin/python3
```

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ ls /usr/lib/python3*
/usr/lib/python3:
dist-packages

/usr/lib/python3.12:
EXTERNALLY-MANAGED                          contextvars.py  mimetypes.py      sqlite3
LICENSE.txt                                 copy.py         modulefinder.py   sre_compile.py
__future__.py                               copyreg.py      multiprocessing   sre_constants.py
__hello__.py                                crypt.py        netrc.py          sre_parse.py
__phello__                                  csv.py          nntplib.py        ssl.py
__pycache__                                 ctypes          ntpath.py         stat.py
_aix_support.py                             curses          nturl2path.py     statistics.py
_collections_abc.py                         dataclasses.py  numbers.py        string.py
_compat_pickle.py                           datetime.py     opcode.py         stringprep.py
_compression.py                             dbm             operator.py       struct.py
_distutils_system_mod.py                    decimal.py      optparse.py       subprocess.py
_markupbase.py                              difflib.py      os.py             sunau.py
_osx_support.py                             dis.py          pathlib.py        symtable.py
_py_abc.py                                  doctest.py      pdb.py            sysconfig.py
_pydatetime.py                              email           pickle.py         tabnanny.py
_pydecimal.py                               encodings       pickletools.py    tarfile.py
_pyio.py                                    enum.py         pipes.py          telnetlib.py
_pylong.py                                  filecmp.py      pkgutil.py        tempfile.py
_sitebuiltins.py                            fileinput.py    platform.py       test
_strptime.py                                fnmatch.py      plistlib.py       textwrap.py
_sysconfigdata__linux_x86_64-linux-gnu.py   fractions.py    poplib.py         this.py
_sysconfigdata__x86_64-linux-gnu.py         ftplib.py       posixpath.py      threading.py
_threading_local.py                         functools.py    pprint.py         timeit.py
_weakrefset.py                              genericpath.py  profile.py        token.py
abc.py                                      getopt.py       pstats.py         tokenize.py
aifc.py                                     getpass.py      pty.py            tomllib
antigravity.py                              gettext.py      py_compile.py     trace.py
argparse.py                                 glob.py         pyclbr.py         traceback.py
ast.py                                      graphlib.py     pydoc.py          tracemalloc.py
asyncio                                     gzip.py         pydoc_data        tty.py
base64.py                                   hashlib.py      queue.py          turtle.py
bdb.py                                      heapq.py        quopri.py         types.py
bisect.py                                   hmac.py         random.py         typing.py
bz2.py                                      html            re                unittest
cProfile.py                                 http            reprlib.py        urllib
calendar.py                                 imaplib.py      rlcompleter.py    uu.py
cgi.py                                      imghdr.py       runpy.py          uuid.py
cgitb.py                                    importlib       sched.py          venv
chunk.py                                    inspect.py      secrets.py        warnings.py
cmd.py                                      io.py           selectors.py      wave.py
code.py                                     ipaddress.py    shelve.py         weakref.py
codecs.py                                   json            shlex.py          webbrowser.py
codeop.py                                   keyword.py      shutil.py         wsgiref
collections                                 lib-dynload     signal.py         xdrlib.py
```

User and Security

Python runs under the current user account, which ensures that programs do not have unnecessary system privileges. This helps improve system security.

Service Management

Python itself is not a service, but Python programs can be configured to run as services using system tools.

Update Model

Updates are provided through the package manager, and security patches are released regularly.

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Part C: FOSS Ecosystem

Open-source software does not exist in isolation. Every major project is part of a larger ecosystem of tools, libraries, contributors, and communities. Python is a strong example of this because its growth has been directly influenced by the ecosystem around it, and at the same time, it has enabled the creation of many other tools.

One of the most important aspects of Python's ecosystem is its dependency structure. Python itself relies on system-level components such as compilers, libraries, and runtime environments. For example, parts of Python are implemented in C, which means it depends on compilers like GCC for building and maintaining the language. In addition to this, Python uses standard libraries that provide built-in functionality for tasks such as file handling, networking, and data processing.

Another important component is the package management system, mainly pip. Pip allows developers to install and manage external libraries easily. This has played a major role in Python's popularity because it allows developers to extend the language without having to build everything from scratch. For example, if someone wants to perform data analysis, they can simply install libraries like NumPy or Pandas instead of writing complex code themselves.

Python has also enabled the development of a large number of frameworks and tools. In web development, frameworks like Django and Flask allow developers to build web applications efficiently. Django is more structured and suited for larger projects, while Flask is lightweight and flexible. In the field of data science and machine learning, libraries like NumPy, Pandas, TensorFlow, and PyTorch have made Python the preferred choice for researchers and developers.

Another important part of the ecosystem is the concept of forks and extensions. Since Python is open source, developers can modify the language or create variations based on it.

This has led to different implementations such as CPython, PyPy, and Jython. Each of these serves a different purpose, such as improving performance or enabling compatibility with other systems.

Python also plays a role in the broader web ecosystem. While it is not traditionally part of the LAMP stack (Linux, Apache, MySQL, PHP), it often replaces PHP in modern applications. Many web servers run Python-based applications, and frameworks like Django integrate well with databases and web servers.

The community is one of the strongest aspects of Python's ecosystem. It includes developers, researchers, educators, and organizations from around the world. Contributions come in many forms, such as writing code, fixing bugs, improving documentation, and helping others in forums. Platforms like GitHub host Python's source code, where developers can collaborate openly.

There are also events and conferences such as PyCon, where developers come together to share knowledge and discuss improvements. The Python Software Foundation plays an important role in managing the development of Python and supporting the community.

Overall, Python's ecosystem shows how open-source software grows through collaboration. It is not just a single tool, but part of a larger network where different components support and improve each other.

Part D: Open Source vs Proprietary

To better understand the value of Python as an open-source software, it is important to compare it with proprietary alternatives. Proprietary software is owned and controlled by a company, and users are typically given limited access to its functionality. In contrast, open-source software like Python provides greater freedom and transparency.

One of the most obvious differences is cost. Python is completely free to use, which makes it accessible to students, developers, and organizations of all sizes. Proprietary software often requires purchasing licenses, which can be expensive and limit access for some users. This difference plays a significant role in education, where students can learn programming using Python without any financial barriers.

Another important factor is transparency. Since Python is open source, its code is available for anyone to inspect. This allows developers to understand how the language works internally and even modify it if needed. In proprietary software, the source code is hidden, which means users have to rely on the company for updates and fixes.

Security is also influenced by this difference. In open-source software, vulnerabilities can be identified and fixed by the community. This can lead to faster detection and resolution of issues. However, it also means that the code is publicly visible, which could potentially be

studied by attackers. In proprietary software, security relies on the company's internal processes, and users have limited visibility into how issues are handled.

Support and reliability are areas where proprietary software sometimes has an advantage. Companies often provide dedicated customer support and guaranteed updates. In open-source software, support is usually community-driven. While Python has a very strong community and extensive documentation, there is no single authority responsible for user support.

Freedom to modify is one of the biggest advantages of Python. Developers can customize it according to their needs, integrate it with other systems, and build their own tools on top of it. Proprietary software usually restricts such modifications, which can limit flexibility.

Another important difference is control. In open-source software, control is distributed among the community. Decisions are made collaboratively, and development is transparent. In proprietary software, control lies with the company, which decides the direction of the software based on its own priorities.

When comparing Python with proprietary programming environments, it becomes clear that Python offers a balance of flexibility, accessibility, and power. It allows users to experiment, learn, and innovate without restrictions.

However, it is also important to recognize that proprietary software is not always inferior. In some cases, it may offer better integration, support, or performance for specific use cases. The choice between open source and proprietary software depends on the requirements of the user.

Dimension	**Python**	**Proprietary Software**
Cost	Free	Paid
Security	Transparent	Closed
Support	Community	Company
Freedom	High	Limited
Control	Community	Corporate

Shell Scripts

Script 1

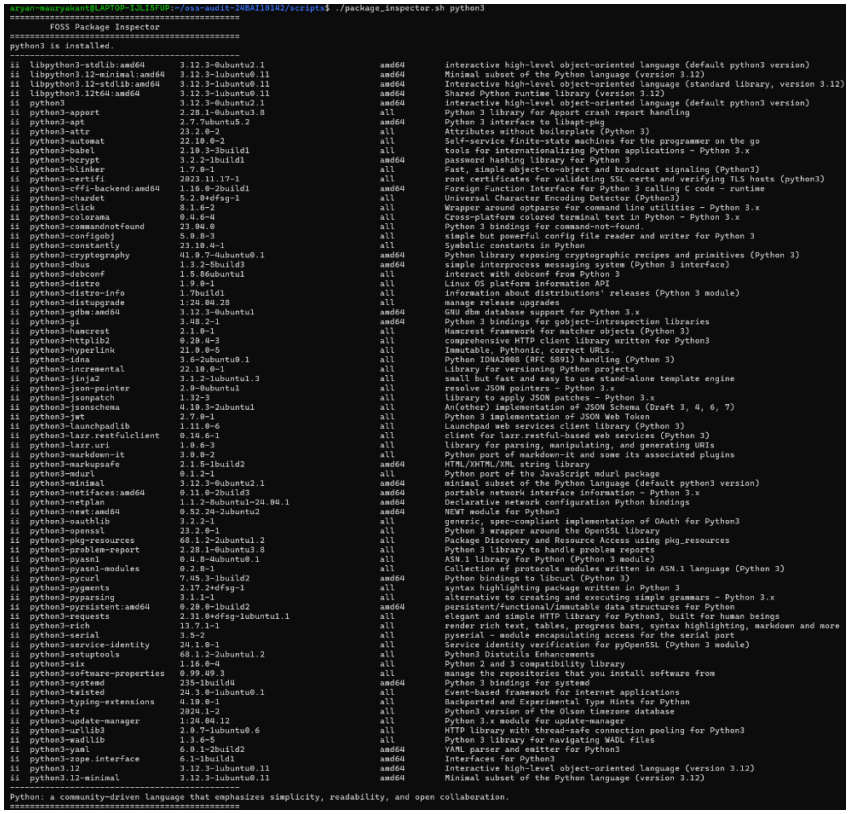
```
#!/bin/bash

# Script 1: System Identity Report

# Author: Aryan Mauryakant | Reg No: 24BAI10142
```

```
# Course: Open Source Software

STUDENT_NAME="Aryan Mauryakant"

REG_NO="24BAI10142"

SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)

USER_NAME=$(whoami)

UPTIME=$(uptime -p)

DATE=$(date '+%d %B %Y, %H:%M:%S')

HOME_DIR=$HOME

DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')

echo "============================================="

echo "   Open Source Audit — System Identity Report"

echo "============================================="

echo "Student Name : $STUDENT_NAME"

echo "Reg Number   : $REG_NO"

echo "Software     : $SOFTWARE_CHOICE"

echo "---------------------------------------------"

echo "OS           : $DISTRO"

echo "Kernel       : $KERNEL"

echo "User         : $USER_NAME"

echo "Home Dir     : $HOME_DIR"

echo "Uptime       : $UPTIME"

echo "Date & Time  : $DATE"

echo "---------------------------------------------"

echo "License Info : This Linux system is based on open-source principles, typically licensed under GPL."

echo "=============================================="
```

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ ./system_identity.sh
=============================================
   Open Source Audit — System Identity Report
=============================================
Student Name : Aryan Mauryakant
Reg Number   : 24BAI10142
Software     : Python
---------------------------------------------
OS           : Ubuntu 24.04.4 LTS
Kernel       : 6.6.87.2-microsoft-standard-WSL2
User         : aryan-mauryakant
Home Dir     : /home/aryan-mauryakant
Uptime       : up 15 minutes
Date & Time  : 30 March 2026, 16:18:17
---------------------------------------------
License Info : This Linux system is based on open-source principles, typically licensed under GPL.
==============================================
```

Script 2

```
#!/bin/bash

# Aryan Mauryakant | 24BAI10142

PACKAGE=$1

[ -z "$PACKAGE" ] && echo "Usage: ./package_inspector.sh <package>" && exit 1

if dpkg -l | grep -qw "$PACKAGE"; then

  echo "$PACKAGE is installed"

  dpkg -l | grep "$PACKAGE"

else

  echo "$PACKAGE is NOT installed"

fi

case $PACKAGE in

  python3) echo "Python: simple, readable, open-source language." ;;

  git) echo "Git: version control for collaborative development." ;;

  vlc) echo "VLC: free media player built by community." ;;

  *) echo "Open-source software promotes collaboration." ;;

esac
```



```
aryan-mauryakant@LAPTOP-I3LI5FUP:~/oss-audit-24BAI10142/scripts$ ./package_inspector.sh python3
==========================================
          FOSS Package Inspector
==========================================
python3 is installed.
------------------------------------------
ii  libpython3-stdlib:amd64    3.12.3-0ubuntu2.1        amd64    interactive high-level object-oriented language (default python3 version)
ii  libpython3.12-minimal:amd64  3.12.3-1ubuntu0.11     amd64    Minimal subset of the Python language (version 3.12)
ii  libpython3.12-stdlib:amd64   3.12.3-1ubuntu0.11     amd64    Interactive high-level object-oriented language (standard library, version 3.12)
ii  libpython3.12t64:amd64      3.12.3-1ubuntu0.11       amd64    Shared Python runtime library (version 3.12)
ii  python3                     3.12.3-0ubuntu2.1        amd64    interactive high-level object-oriented language (default python3 version)
ii  python3-apport              2.28.1-0ubuntu3.8        all      Python 3 library for Apport crash report handling
ii  python3-apt                 2.7.7ubuntu5.2           amd64    Python 3 interface to libapt-pkg
ii  python3-attr                23.2.0-2                 all      Attributes without boilerplate (Python 3)
ii  python3-automat             22.10.0-2                all      Self-service finite-state machines for the programmer on the go
ii  python3-babel               2.10.3-3build1           all      tools for internationalizing Python applications - Python 3.x
ii  python3-bcrypt              3.2.2-1build1            amd64    password hashing library for Python 3
ii  python3-blinker             1.7.0-1                  all      Fast, simple object-to-object and broadcast signaling (Python3)
ii  python3-certifi             2023.11.17-1             all      root certificates for validating SSL certs and verifying TLS hosts (python3)
ii  python3-cffi-backend:amd64  1.16.0-2build1           amd64    Foreign Function Interface for Python 3 calling C code - runtime
ii  python3-chardet             5.2.0+dfsg-1             all      Universal Character Encoding Detector (Python3)
ii  python3-click               8.1.6-2                  all      Wrapper around optparse for command line utilities - Python 3.x
ii  python3-colorama            0.4.6-4                  all      Cross-platform colored terminal text in Python - Python 3.x
ii  python3-commandnotfound     23.04.0                  all      Python 3 bindings for command-not-found.
ii  python3-configobj           5.0.8-3                  all      simple but powerful config file reader and writer for Python 3
ii  python3-constantly          23.10.4-1                all      Symbolic constants in Python
ii  python3-cryptography        41.0.7-4ubuntu0.1        amd64    Python library exposing cryptographic recipes and primitives (Python 3)
ii  python3-dbus                1.3.2-5build3            amd64    simple interprocess messaging system (Python 3 interface)
ii  python3-debconf             1.5.86ubuntu1            all      interact with debconf from Python 3
ii  python3-distro              1.9.0-1                  all      Linux OS platform information API
ii  python3-distro-info         1.7build1                all      information about distributions' releases (Python 3 module)
ii  python3-distupgrade         1:24.04.28               all      manage release upgrades
ii  python3-gdbm:amd64          3.12.3-0ubuntu1          amd64    GNU dbm database support for Python 3.x
ii  python3-gi                  3.48.2-1                 amd64    Python 3 bindings for gobject-introspection libraries
ii  python3-hamcrest            2.1.0-1                  all      Hamcrest framework for matcher objects (Python 3)
ii  python3-httplib2            0.20.4-3                 all      comprehensive HTTP client library written for Python3
ii  python3-hyperlink           21.0.0-5                 all      Immutable, Pythonic, correct URLs.
ii  python3-idna                3.6-2ubuntu0.1           all      Python IDNA2008 (RFC 5891) handling (Python 3)
ii  python3-incremental         22.10.0-1                all      Library for versioning Python projects
ii  python3-jinja2              3.1.2-1ubuntu1.3         all      small but fast and easy to use stand-alone template engine
ii  python3-json-pointer        2.0-0ubuntu1             all      resolve JSON pointers - Python 3.x
ii  python3-jsonpatch           1.32-3                   all      library to apply JSON patches - Python 3.x
ii  python3-jsonschema          4.19.3-2ubuntu1          all      An(other) implementation of JSON Schema (Draft 3, 4, 6, 7)
ii  python3-jwt                 2.7.0-1                  all      Python 3 implementation of JSON Web Token
ii  python3-launchpadlib        1.11.0-6                 all      Launchpad web services client library (Python 3)
ii  python3-lazr.restfulclient  0.14.6-1                 all      client for lazr.restful-based web services (Python 3)
ii  python3-lazr.uri            1.0.6-3                  all      library for parsing, manipulating, and generating URIs
ii  python3-markdown-it         3.0.0-2                  all      Python port of markdown-it and some its associated plugins
ii  python3-markupsafe          2.1.5-1build2            amd64    HTML/XHTML/XML string library
ii  python3-mdurl               0.1.2-1                  all      Python port of the JavaScript mdurl package
ii  python3-minimal             3.12.3-0ubuntu2.1        amd64    minimal subset of the Python language (default python3 version)
ii  python3-netifaces:amd64     0.11.0-2build3           amd64    portable network interface information - Python 3.x
ii  python3-netplan             1.1.2-0ubuntu1~24.04.1   amd64    Declarative network configuration Python bindings
ii  python3-newt:amd64          0.52.24-2ubuntu2         amd64    NEWT module for Python3
ii  python3-oauthlib            3.2.2-1                  all      generic, spec-compliant implementation of OAuth for Python3
ii  python3-openssl             23.2.0-1                 all      Python 3 wrapper around the OpenSSL library
ii  python3-pkg-resources       68.1.2-2ubuntu1.2        all      Package Discovery and Resource Access using pkg_resources
ii  python3-problem-report      2.20.1-0ubuntu3.8        all      Python 3 library to handle problem reports
ii  python3-pyasn1              0.4.8-4ubuntu0.1         all      ASN.1 library for Python (Python 3 module)
ii  python3-pyasn1-modules      0.2.8-1                  all      Collection of protocols modules written in ASN.1 language (Python 3)
ii  python3-pycurl              7.45.3-1build2           amd64    Python bindings to libcurl (Python 3)
ii  python3-pygments            2.17.24dfsg-1            all      syntax highlighting package written in Python 3
ii  python3-pyparsing           3.1.1-1                  all      alternative to creating and executing simple grammars - Python 3.x
ii  python3-pyrsistent:amd64    0.20.0-1build2           amd64    persistent/functional/immutable data structures for Python
ii  python3-requests            2.31.0+dfsg-1ubuntu1.1   all      elegant and simple HTTP library for Python3, built for human beings
ii  python3-rich                13.7.1-1                 all      render rich text, tables, progress bars, syntax highlighting, markdown and more
ii  python3-serial              3.5-2                    all      pyserial - module encapsulating access for the serial port
ii  python3-service-identity    24.1.0-1                 all      Service identity verification for pyOpenSSL (Python 3 module)
ii  python3-setuptools          68.1.2-2ubuntu1.2        all      Python3 Distutils Enhancements
ii  python3-six                 1.16.0-4                 all      Python 2 and 3 compatibility library
ii  python3-software-properties 0.99.49.3                all      manage the repositories that you install software from
ii  python3-systemd             235-1build4              amd64    Python 3 bindings for systemd
ii  python3-twisted             24.3.0-1ubuntu0.1        all      Event-based framework for internet applications
ii  python3-typing-extensions   4.10.0-1                 all      Backported and Experimental Type Hints for Python
ii  python3-tz                  2024.1-2                 all      Python3 version of the Olson timezone database
ii  python3-update-manager      1:24.04.12               all      Python 3.x module for update-manager
ii  python3-urllib3             2.0.7-1ubuntu0.6         all      HTTP library with thread-safe connection pooling for Python3
ii  python3-wadllib             1.3.6-5                  all      Python 3 library for navigating WADL files
ii  python3-yaml                6.0.1-2build2            amd64    YAML parser and emitter for Python3
ii  python3-zope.interface      6.1-1build1              amd64    Interfaces for Python3
ii  python3.12                  3.12.3-1ubuntu0.11       amd64    Interactive high-level object-oriented language (version 3.12)
ii  python3.12-minimal          3.12.3-1ubuntu0.11       amd64    Minimal subset of the Python language (version 3.12)
------------------------------------------
Python: a community-driven language that emphasizes simplicity, readability, and open collaboration.
==========================================
```

Script 3

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

# Aryan Mauryakant | 24BAI10142

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld "$DIR" | awk '{print $1,$3,$4}')

    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

    echo "$DIR => $PERMS | $SIZE"

  else

    echo "$DIR not found"

  fi

done
```

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ ./disk_auditor.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 17M
/home => Permissions: drwxr-xr-x root root | Size: 64K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K
```

Script 4

```
#!/bin/bash

# Script 4: Log File Analyzer

# Aryan Mauryakant | 24BAI10142

LOGFILE=$1

KEYWORD=${2:-error}

COUNT=0

[ ! -f "$LOGFILE" ] && echo "File not found" && exit 1

while read -r LINE; do

  echo "$LINE" | grep -iq "$KEYWORD" && COUNT=$((COUNT+1))

done < "$LOGFILE"
```

echo "'$KEYWORD' found $COUNT times in $LOGFILE"

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ sudo ./log_analyzer.sh /var/log/syslog
[sudo] password for aryan-mauryakant:
Keyword 'error' found 7 times in /var/log/syslog
```

Script 5

#!/bin/bash

 # Script 5: Open Source Manifesto Generator

 # Aryan Mauryakant | 24BAI10142

read -p "Tool: " TOOL

read -p "Freedom (one word): " FREEDOM

read -p "Build and share: " BUILD

OUT="manifesto_$(whoami).txt"

echo "I use $TOOL daily. For me, freedom means $FREEDOM. I would build $BUILD and share it openly." > "$OUT"

echo "Saved to $OUT"

cat "$OUT"

```
aryan-mauryakant@LAPTOP-IJLI5FUP:~/oss-audit-24BAI10142/scripts$ ./manifesto.sh
Answer the following questions:

1. Name one open-source tool you use every day: Python3
2. In one word, what does 'freedom' mean to you? maturity
3. Name one thing you would build and share freely: Portraits

Manifesto saved to manifesto_aryan-mauryakant.txt

Open Source Manifesto
Date: 30 March 2026

I use Python3 in my daily life, and it shows how open-source software can make technology accessible to everyone.
To me, freedom means maturity. This idea is important because it allows people to learn, modify, and share software withou
t restrictions.
If given a chance, I would build Portraits and share it openly so others can benefit from it.
```

Conclusion

This project helped me understand open-source software from both a technical and a practical point of view. Before working on this, I mostly saw tools like Python as just something we use to write programs or complete assignments. I did not really think about where they came from, who maintains them, or what makes them different from other types of software. Going through this audit changed that perspective.

By studying Python in detail, I was able to see how a simple idea—making programming easier and more readable—can grow into something that is used all over the world. What stood out to me the most was how Python was not built by a single company with a fixed goal, but instead improved over time by a community of developers. This explains why it is so flexible and widely used across different fields like web development, data science, and artificial intelligence.

The licensing part of the project also made a big difference in my understanding. Earlier, I used to think “free software” simply meant software that does not cost money. Now I understand that it is more about freedom—freedom to use, study, modify, and share. Python’s license allows this level of freedom, which is one of the reasons why it has been adopted so widely, including by large companies.

Working with Linux during this project gave me a more practical understanding of how software actually exists on a system. Instead of just writing code, I had to look at directories, permissions, installation methods, and system behaviour. Writing the shell scripts was also useful because it showed how small tools can automate tasks and interact directly with the system. Even though the scripts were simple, they helped connect theory with real usage.

Another important takeaway for me was understanding the ecosystem around Python. It is not just a language by itself, but part of a much larger network of tools, libraries, and frameworks. This made me realize how open-source projects depend on each other and grow together rather than in isolation.

Overall, this project made me think more seriously about how software is developed and shared. Python is a strong example of how open-source principles can lead to something reliable, widely used, and continuously improving. Personally, I would choose Python for most projects because of its simplicity and flexibility. I would also consider contributing to open-source projects in the future, even in a small way, because it seems like a good way to learn and be part of something larger.

In conclusion, this project was not just about completing an assignment. It gave me a better understanding of how open-source software works in the real world, both technically and conceptually, and that is something I will carry forward in my future work.